

AI-AUGMENTED BACKEND DEVELOPMENT WITH NODE.JS AND MONGODB

TEAM DETAILS

Team Leader	MOGANA SHREE K S
--------------------	------------------

TEAM MEMBERS

MEMBERS	NAME	ROLE
Member 1	MOGANA SHREE K S	Code execution
Member 2	MONISHA N	Demo video
Member 3	MONCY P	PPT
Member 4	MOHANAPRIYA R	Documentation

PROJECT TITLE: AI FAQ ASSISTANT

1. INTRODUCTION

The AI-Augmented FAQ Assistant is a backend-oriented application developed under the AI-Augmented Backend Development technology track. The project focuses on combining conventional Frequently Asked Questions management with Artificial Intelligence-based assistance.

Traditional FAQ systems normally depend on predefined questions and answers. Although such systems are useful for common problems, they may become difficult to maintain when the number of questions increases. Users may also ask the same question using different words, making exact keyword matching less effective.

The proposed AI-Augmented FAQ Assistant provides a structured backend through which users can authenticate, search FAQ content, manage FAQ records according to their permissions, and obtain AI-assisted answers. The backend manages the communication between the client application, database, authentication layer, and external Generative AI service.

The system is designed to provide a controlled and secure environment for FAQ creation and intelligent question answering. It separates application logic from frontend presentation and keeps sensitive configuration such as authentication secrets and AI API credentials on the server side.

Overall, the project demonstrates how RESTful backend technologies and AI services can be integrated to create a practical support automation platform

2. PROJECT OVERVIEW

The AI-Augmented FAQ Assistant is an intelligent REST API platform designed to support FAQ creation, management, search, authentication, and AI-powered assistance. The backend receives HTTP requests, validates user input, performs authentication and authorization checks, communicates with MongoDB through Mongoose, and invokes the AI service when intelligent content generation or assistance is required.

The project follows a modular architecture so that authentication, FAQ operations, AI operations, database operations, and error handling can be maintained independently. This approach improves readability, testing, maintenance, and future scalability.

The application can support two major interaction patterns. First, public users can consume published FAQ information and perform permitted searches. Second, authenticated users or content creators can perform protected operations such as creating FAQs and requesting AI-assisted FAQ generation.

The project also demonstrates the use of JWT-based authentication, password hashing, schema validation, RESTful routing, middleware, and AI service integration.

3. KEY OBJECTIVES

- To develop a structured REST API for FAQ and customer-support operations.
- To integrate Generative AI capabilities into the FAQ workflow.
- To provide fast and convenient FAQ search.
- To support natural-language question processing.
- To reduce repetitive manual support activities.
- To allow authorized users to create and manage FAQ content.
- To implement secure authentication and authorization.
- To maintain structured FAQ categories.
- To validate data before database persistence.
- To provide consistent API responses and centralized error handling.
- To demonstrate practical AI-augmented backend development.

4. TARGET USERS

- Customers / Public Users – For viewing published FAQs and searching for common information.
- Authenticated Users – For accessing permitted personalized and AI-assisted functionality.
- Content Creators – For creating, updating, deleting, and organizing FAQ content.
- Administrators – For system-level user, FAQ, category, and AI-content management.
- Support Teams – For reducing repetitive question-answering workload.
- Organizations – For maintaining centralized and searchable support documentation.

5. PROJECT INSTRUCTIONS

1. Install Node.js and npm before beginning backend development.
2. Create a clear project structure containing routes, controllers, models, middleware, services, utilities, and configuration.
3. Use environment variables for sensitive information.
4. Validate incoming request data before database operations.
5. Hash passwords before storing them.
6. Protect private endpoints using JWT authentication.
7. Apply role-based authorization where administrative access is required.
8. Connect the backend to MongoDB using Mongoose.
9. Keep AI communication inside a dedicated service or utility module.
10. Use Postman or Thunder Client to test API requests and responses.
11. Maintain documentation for major APIs and system modules.

6. PRE-REQUISITES

SOFTWARE	PURPOSE
Node.js v18 or above	JavaScript runtime for backend execution
npm v9 or above	Package manager for backend dependencies
MongoDB	NoSQL database for application data
Mongoose	ODM for MongoDB schemas and database operations
Express.js	Backend web framework for REST API development
Visual Studio Code	Development environment
Postman / Thunder Client	REST API testing
Google Gemini / Generative AI API	AI-powered FAQ generation and assistance
Git / GitHub (optional)	Source-code version control

7. KNOWLEDGE REQUIREMENTS

- JavaScript ES6 concepts
- Node.js and Express.js
- REST API development
- HTTP methods and status codes
- MongoDB and Mongoose
- JWT authentication
- Password hashing with bcrypt
- Middleware architecture
- MVC-style application structure
- JSON request and response handling
- Basic Generative AI and API concepts
- Environment-variable configuration

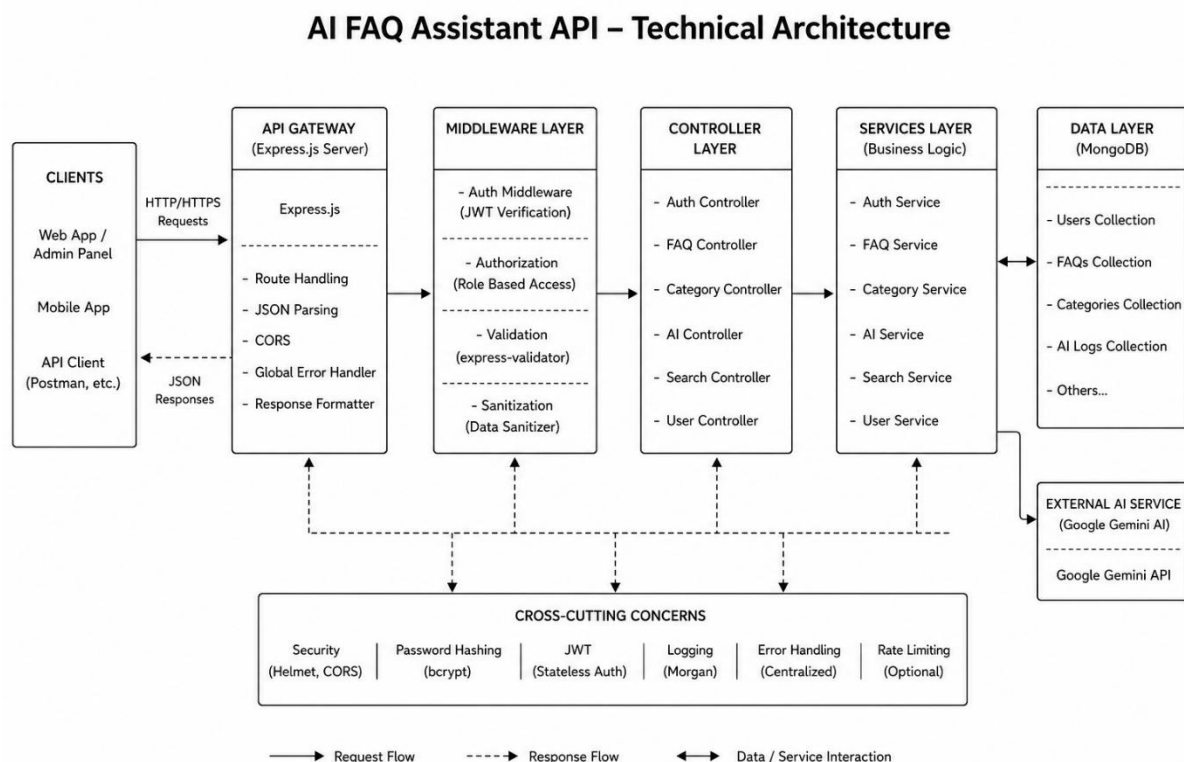
8. SOFTWARE REQUIREMENTS

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js (v18 or above): Runtime environment for executing server-side JavaScript.
- npm (v9 or above): Package manager used to install and manage dependencies.
- Express.js: Web framework used to construct RESTful API routes and middleware.
- MongoDB: NoSQL database used to store user, FAQ, category, and related data.
- Mongoose ODM: Provides schemas, validation, references, and database operations.
- JWT: Provides stateless token-based authentication.
- bcrypt/bcryptjs: Used to hash passwords before database storage.
- Google Gemini / Generative AI SDK: Used for AI-powered FAQ generation and assistance.
- Postman / Thunder Client: Used to verify API requests, responses, authentication, and validation.
- Visual Studio Code: Development and source-code editing environment.

9. HARDWARE REQUIREMENTS

- Processor: Intel Core i5 8th Generation or above / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum; 16 GB recommended for running database and development services concurrently.
- Storage: At least 1 GB of available workspace for source code, dependencies, logs, and project files.
- Internet Connection: Required when the application communicates with a cloud-hosted Generative AI service or remote database.
- Display: Standard monitor capable of running the development environment and API testing tools.

10. Project Architecture



The AI FAQ Assistant application exposes structured endpoints handled by a layered server architecture model. Incoming requests undergo filter processing before interacting with data services.

10.1 Components Definition

- Interface: API tools like Postman query distinct route paths to extract JSON entities.
- Express Server Gateway: Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses incoming JSON payloads, and handles CORS headers.
- Authentication Middleware: Extracts tokens from headers, validates user context signatures, and evaluates authorization thresholds for guarded resource paths.
- Granular Services (Logic Modules): Enforces isolated domain rules for user session life cycles and operational query modifications.
- Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.
- AI Flow (Service Integration): Connected to external AI models via Google Gemini SDK integrations, letting the controller layer offload complex question-answering tasks and automated topic generation workflows.

10.2 Architecture Benefits

- Separation of responsibilities
- Secure protected routes
- Reusable API structure
- Easy database interaction
- Independent AI integration
- Centralized error handling

11. System Modules

11.1 Authentication Module

Manages registration, login, password protection, JWT token generation and authentication verification.

11.2 FAQ Management Module

Provides complete CRUD operations: Create, Read, Update and Delete FAQ records.

11.3 Search Module

Allows users to search FAQ content using keywords and supports faster retrieval of relevant information.

11.4 Category Module

Organizes FAQ records into structured categories for easier navigation and management.

11.5 AI Generation Module

Accepts a topic and requests AI-generated FAQ content from Google Gemini.

11.6 Error Handling Module

Provides standardized API error responses and helps prevent unhandled errors from exposing internal information.

11.7 Module Interaction

A user authenticates through the Authentication Module. Authorized requests can reach FAQ and AI modules. FAQ data is stored through the Mongoose database layer, while AI requests are processed through the Gemini integration.

12. Database Design

MongoDB is used as the primary NoSQL database. Mongoose defines document structures, validation rules and references.

12.1 User Entity

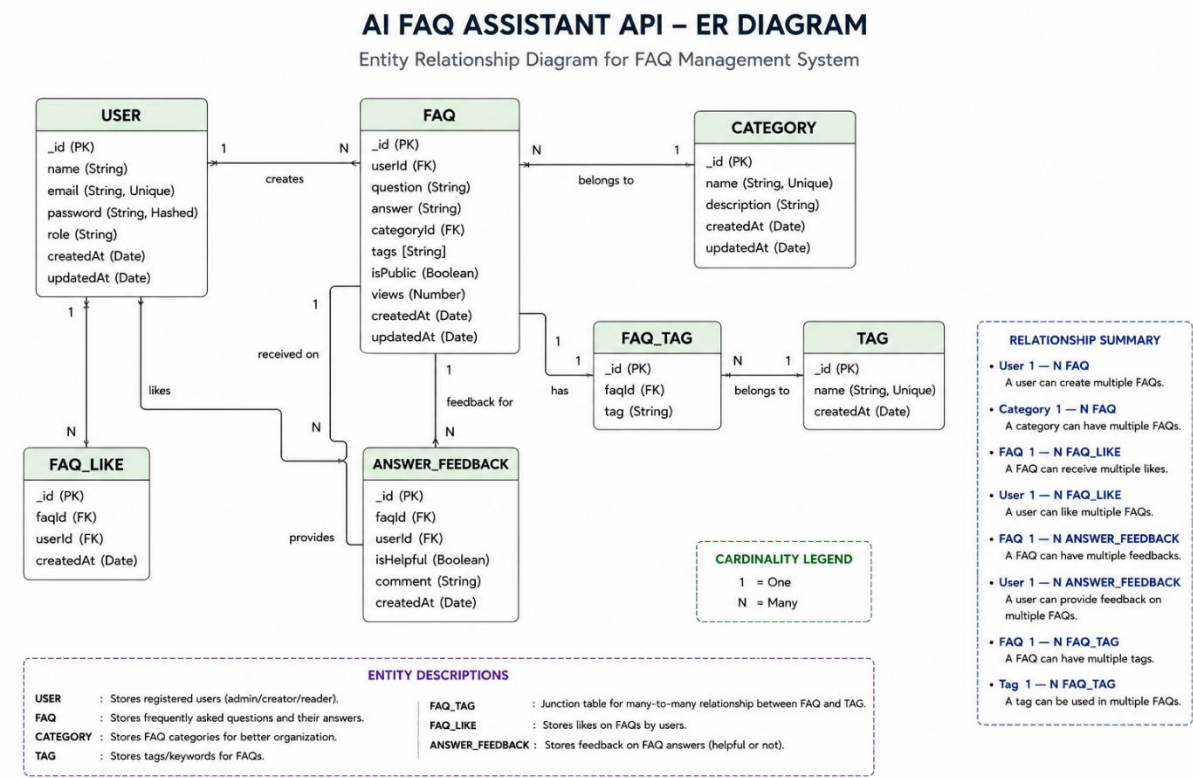
Field	Type	Description
_id	ObjectId	Unique user identifier
name	String	User name
email	String	Unique email
password	String	Hashed password
role	String	User role
createdAt	Date	Creation time
updatedAt	Date	Update time

12.2 FAQ Entity

Field	Type	Description
_id	ObjectId	FAQ identifier
question	String	FAQ question
answer	String	FAQ answer
category	String	FAQ category
createdBy	ObjectId	Reference to creator
createdAt	Date	Creation time
updatedAt	Date	Update time

12.3 Relationships

A registered user can create and manage multiple FAQ records. A category can contain multiple FAQ documents. A user can also generate multiple AI-assisted FAQs.



13. User Roles and Access Control

13.1 Administrator

- Manage registered users.
- View, update and delete FAQs.
- Monitor AI-generated content.
- Manage FAQ categories.
- Access complete system functionality.

13.2 Content Creator

- Create new FAQs.
- Edit personal FAQ records.
- Delete owned FAQs.
- Generate FAQs using AI.
- Organize FAQs under categories.

13.3 Authenticated User

- View public FAQs.
- Search FAQs using keywords.
- Generate AI-assisted answers.
- Access personal profile information.

13.4 Public User

- View published FAQs.
- Search FAQs.
- Cannot create, edit or delete FAQ records.
- No access to protected API endpoints.

13.5 Authorization Flow

The client sends a login request and receives a JWT token. For protected operations, the client sends the token using the Bearer Authorization header. The authentication middleware validates the token before allowing access.

14. Project Setup and Installation

14.1 Creating the Project Folder

Create a project folder named AI-FAQ-Assistant. Inside it create a src folder and open the project in Visual Studio Code.

14.2 Initialize Node.js

Run the following command in the project terminal:

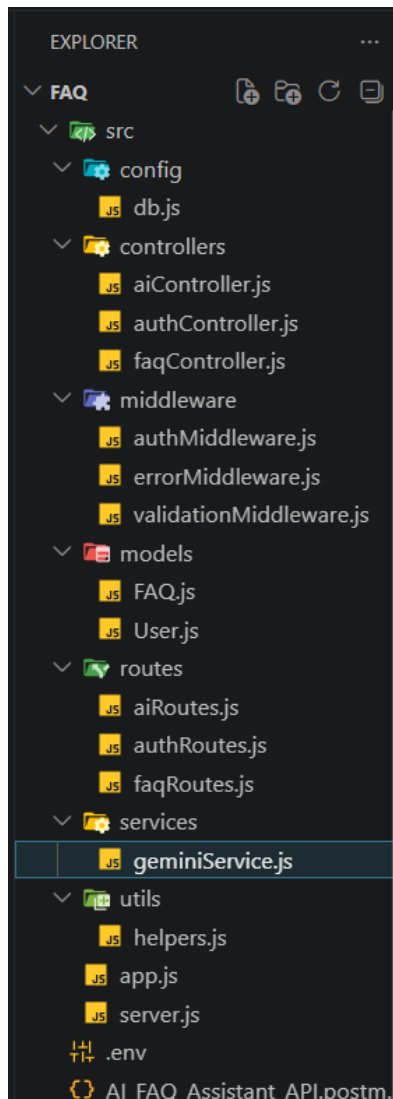
```
npm init -y
```

14.3 Install Dependencies

```
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

14.4 Project Folder Structure

AI-FAQ-Assistant



14.5 Environment Configuration

Database connection information, JWT secret and Gemini API credentials should be stored in environment variables rather than directly in source files.

15. Backend Implementation

15.1 Server Configuration

```
c > server.js > ...
1 const app = require('./app');
2 const connectDB = async () => {
3   try {
4     const dbConnect = require('./config/db');
5     await dbConnect();
6   } catch (error) {
7     console.error('Failed to connect to the database', error);
8   }
9 };
10
11 const startServer = async () => {
12   // Connect to database
13   await connectDB();
14
15   const PORT = process.env.PORT || 5000;
16
17   const server = app.listen(PORT, () => {
18     console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
19   });
20
21   // Handle unhandled promise rejections
22   process.on('unhandledRejection', (err, promise) => {
23     console.error('Unhandled Rejection: ${err.message}');
24     // Close server & exit process
25     server.close(() => process.exit(1));
26   });
27 };
28
29 startServer();
30
```

server.js acts as the main entry point. It loads environment variables, establishes the database connection and starts the Express server on the configured port.

15.2 Express Application

```
src > app.js > ...
1 const express = require('express');
2 const cors = require('cors');
3 const dotenv = require('dotenv');
4 const { errorHandler } = require('./middleware/errorMiddleware');
5
6 // Load environment variables
7 dotenv.config();
8
9 const app = express();
10
11 // Enable CORS
12 app.use(cors());
13
14 // Body parser middlewares
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: false }));
17
18 // Basic status check route
19 app.get('/', (req, res) => {
20   res.json({
21     success: true,
22     message: 'AI FAQ Assistant API is running.',
23     version: '1.0.0',
24   });
25 });
26
27 // Register API Routes
28 app.use('/api/auth', require('./routes/authRoutes'));
29 app.use('/api/faqs', require('./routes/faqRoutes'));
30 app.use('/api/ai', require('./routes/aiRoutes'));
31
32 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
33 app.use('/socket.io', (req, res) => {
34   res.status(404).json({ error: 'Socket.IO not enabled' });
35 });
36
```

app.js configures Express middleware such as CORS and JSON body parsing, and connects the API route modules.

15.3 Authentication Middleware

```
> middleware > authMiddleware.js > ...
1 const jwt = require('jsonwebtoken');
2 const User = require('../models/User');
3
4 const protect = async (req, res, next) => {
5   let token;
6
7   if (
8     req.headers.authorization &&
9     req.headers.authorization.startsWith('Bearer')
10  ) {
11    try {
12      // Get token from header
13      token = req.headers.authorization.split(' ')[1];
14
15      // Verify token
16      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret_key_here_must_be_long_and_secure');
17
18      // Get user from the token, exclude password
19      req.user = await User.findById(decoded.id).select('-password');
20
21      if (!req.user) {
22        return res.status(401).json({
23          success: false,
24          message: 'Not authorized, user not found',
25        });
26      }
27
28      next();
29    } catch (error) {
30      console.error(error);
31      return res.status(401).json({
32        success: false,
33        message: 'Not authorized, token failed',
34      });
35    }
36  }
37 }
```

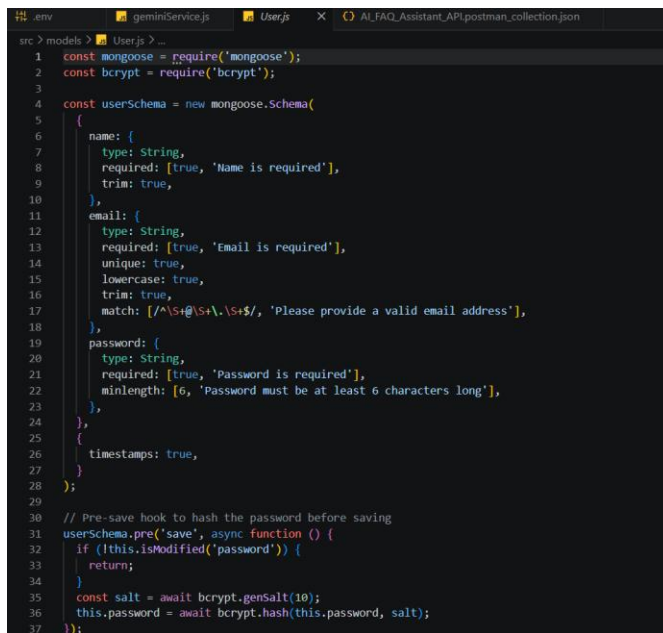
authMiddleware.js intercepts protected requests, extracts the authorization token, validates the JWT signature and establishes the authenticated user context.

15.4 Error Middleware

```
> middleware > errorMiddleware.js > ...
1 const errorHandler = (err, req, res, next) => {
2   let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
3   let message = err.message;
4
5   // Respect status code set on the error object (e.g., 429 from Gemini SDK or 400/403/404)
6   if (err.status) {
7     statusCode = err.status;
8   }
9
10  // Mongoose duplicate key error (e.g. duplicate email)
11  if (err.code === 11000) {
12    statusCode = 400;
13    const field = Object.keys(err.keyValue)[0];
14    message = `Duplicate field value entered: ${field}. Please use another value.`;
15  }
16
17  // Mongoose validation error
18  if (err.name === 'ValidationError') {
19    statusCode = 400;
20    message = Object.values(err.errors)
21      .map((val) => val.message)
22      .join(', ');
23  }
24
25  // Mongoose bad ObjectId
26  if (err.name === 'CastError') {
27    statusCode = 404;
28    message = `Resource not found with id of ${err.value}`;
29  }
30
31  // Only log stack trace to the console for unexpected 500 Server Errors
32  if (statusCode === 500) {
33    console.error(err.stack);
34  } else {
35  }
```

errorMiddleware.js centralizes error responses so that API clients receive consistent error information.

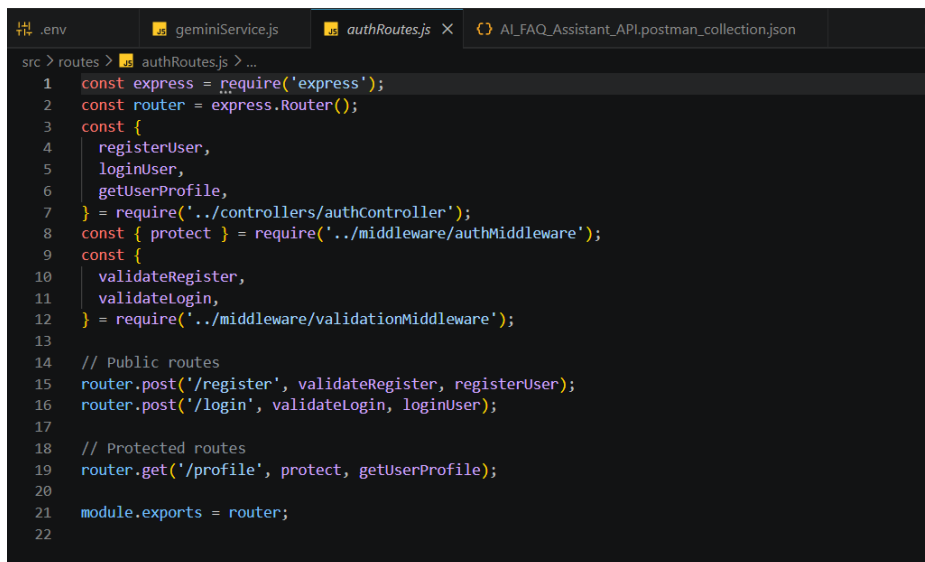
15.5 Mongoose Models



```
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcrypt');
3
4 const userSchema = new mongoose.Schema(
5   {
6     name: {
7       type: String,
8       required: [true, 'Name is required'],
9       trim: true,
10    },
11    email: {
12      type: String,
13      required: [true, 'Email is required'],
14      unique: true,
15      lowercase: true,
16      trim: true,
17      match: [/^\s+@(\s+\.\s+)$/, 'Please provide a valid email address'],
18    },
19    password: {
20      type: String,
21      required: [true, 'Password is required'],
22      minlength: [6, 'Password must be at least 6 characters long'],
23    },
24  },
25  {
26    timestamps: true,
27  }
28 );
29
30 // Pre-save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });
```

User.js defines the user document structure and password protection behavior. FAQ.js defines question, answer, category and creator reference fields.

15.6 API Routes



```
1 const express = require('express');
2 const router = express.Router();
3
4 {
5   registerUser,
6   loginUser,
7   getUserProfile,
8 } = require('../controllers/authController');
9 const { protect } = require('../middleware/authMiddleware');
10 const {
11   validateRegister,
12   validateLogin,
13 } = require('../middleware/validationMiddleware');
14
15 // Public routes
16 router.post('/register', validateRegister, registerUser);
17 router.post('/login', validateLogin, loginUser);
18
19 // Protected routes
20 router.get('/profile', protect, getUserProfile);
21
22 module.exports = router;
```

authRoutes.js handles registration and login.

faqRoutes.js handles FAQ operations.

aiRoutes.js handles AI-generated FAQ requests.

16. REST API Endpoints

16.1 User Registration

Method: POST | Endpoint: /api/auth/register | Access: Public

Example:

```
{"name":"PriyaSharma","email":"priya@writeflow.com","password":"securepassword123"}
```

16.2 User Login

Method: POST | Endpoint: /api/auth/login | Access: Public

Example: {"email":"priya@writeflow.com","password":"securepassword123"}

16.3 Create FAQ

Method: POST | Endpoint: /api/faqs | Access: Private (Bearer JWT)

Example: {"question":"How do I configure custom category tags?","answer":"Navigate to settings panel and save the category configuration.","category":"Configuration"}

16.4 Search FAQ

Method: GET | Endpoint: /api/faqs/search?q=configure | Access: Public

16.5 Generate FAQ Using AI

Method: POST | Endpoint: /api/ai/generate-faq | Access: Private (Bearer JWT)

Example: {"topic":"Mongoose schema validation"}

16.6 CRUD Operations

The FAQ service supports creation, retrieval, updating and deletion of FAQ records. Protected operations require appropriate authentication and authorization.

17. AI Integration and Security

17.1 Google Gemini Integration

The AI module integrates Google Gemini to generate FAQ content dynamically. A user provides a topic, and the service requests generated question, answer and category information from the AI model.

17.2 AI Generation Flow

User → Authenticated AI Endpoint → Topic Validation → Gemini AI → Generated FAQ Content → Response / Optional Persistence

17.3 JWT Security

JSON Web Tokens provide stateless authentication for protected API endpoints. After login, the token is supplied in the Authorization header as a Bearer token.

17.4 Password Security

Passwords are hashed using bcrypt before storage. Plain-text passwords should not be stored in the database.

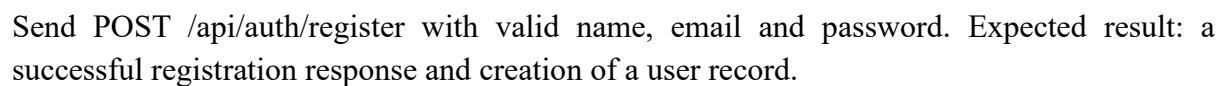
17.5 Schema Validation

Mongoose validation ensures that required fields and expected data structures are checked before persistence.

17.6 Environment Security

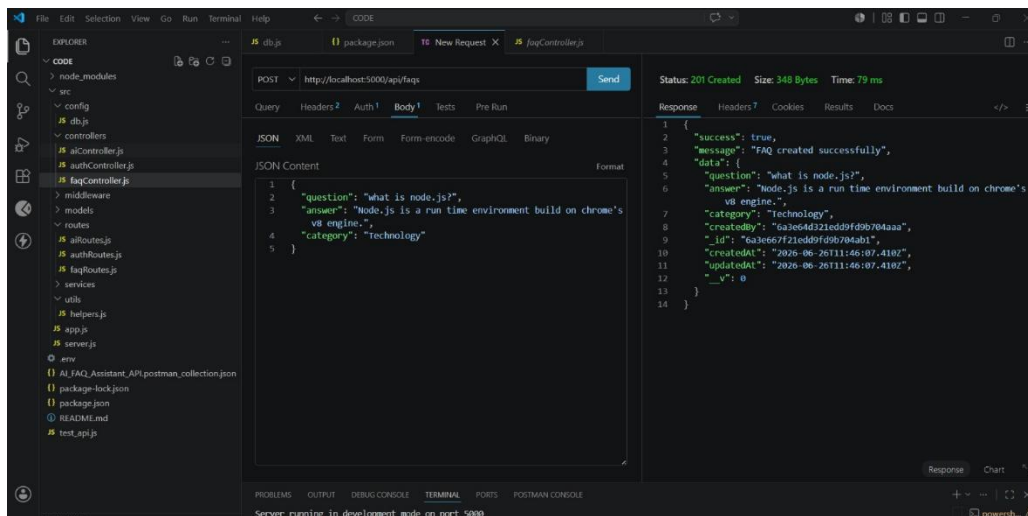
Sensitive values such as database credentials, JWT secrets and AI API keys should be stored in a .env file and excluded from source-code sharing and public repositories.

18.1 Registration Test

[illegible]

Send POST `/api/auth/login` with registered credentials. Expected result: a successful response containing an authentication token.

18.3 Create FAQ Test

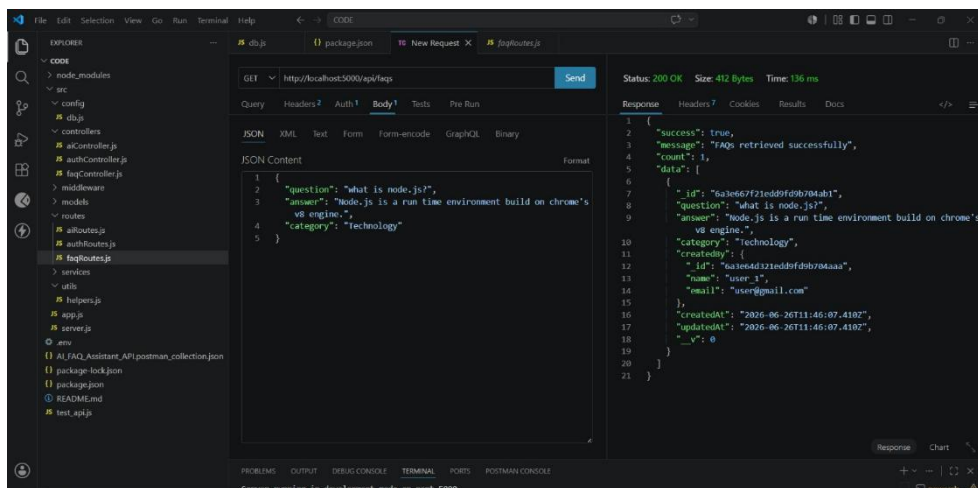


Send POST `/api/faqs` with a valid Bearer JWT token and FAQ JSON body. Expected result: the FAQ is stored in MongoDB.

18.4 Search Test

Send GET `/api/faqs/search?q=configure`. Expected result: matching FAQ documents are returned.

18.5 AI Generation Test



Send POST `/api/ai/generate-faq` with a valid JWT and a topic. Expected result: generated FAQ content is returned.

18.6 Testing Tools

Postman or Thunder Client can be used to execute requests, add authorization headers, provide JSON bodies and inspect HTTP status codes and responses.

18.7 Validation Checklist

- Registration works with valid data.
- Login returns an authentication token.
- Protected routes reject missing or invalid tokens.
- FAQ records are stored correctly.
- Search returns matching records.
- AI endpoint returns generated content.
- Invalid input produces controlled error responses.

19. Conclusion and Future Enhancements

19.1 Conclusion

The AI FAQ Assistant provides a centralized backend platform for intelligent FAQ and customer support management. It combines REST API architecture, MongoDB database management, secure authentication, FAQ CRUD operations, search functionality, categorization and AI-assisted content generation.

Node.js and Express.js provide the backend framework, while MongoDB and Mongoose support flexible document storage and schema validation. JWT and bcrypt provide authentication and password protection. Google Gemini adds AI-assisted content generation to reduce manual FAQ creation effort.

19.2 Future Enhancements

- Complete web-based frontend.
- Advanced semantic or vector search.
- FAQ analytics dashboard.
- Multilingual FAQ generation.
- Automated FAQ quality checking.
- User feedback and rating system.
- Email notification functionality.
- Advanced administrator dashboard.

- FAQ usage statistics.
- Cloud deployment.
- Automated backup and recovery.
- More detailed role and permission management.
-

19.3 Final Outcome

- Secure authentication
- FAQ management
- AI-assisted content generation
- Search functionality
- Category organization
- MongoDB persistence
- RESTful API architecture
- Error handling
- Schema validation.